Reviewer Pack — NW-Design Restriction Dimension Separates Perm from Padded D…

Entry 0cac01ce9ab0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 04:25:13 UTC

NW-Design Restriction Dimension Separates Perm from Padded Det

Entry ID: `0cac01ce9ab0` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-18 04:25:13 UTC

1. Conjecture

Field A (mathematical branch): Nisan–Wigderson combinatorial designs (set systems with bounded pairwise intersection from PRG construction; Nisan–Wigderson 1994; Impagliazzo–Wigderson 1997) **Field B** (complexity object): Mulmuley–Sohoni GCT determinantal complexity dc(perm_n): orbit-closure separation of perm_n from padded det_m for m ≤ n^{1.5} (Mignon–Ressayre 2004; Landsberg 2017)

Statement:

> Let D=(S_1,…,S_M) be a Nisan–Wigderson design on the index set [n]×[n] of matrix-entry variables x_{ij}, with each |S_t|=n and pairwise intersections |S_s∩S_t|≤k=⌈log₂ n⌉. For a homogeneous degree-n polynomial f∈ℂ[x_{ij}]*n, define the NW-restriction dimension ρ_D(f) := dim_ℂ span{ f^{(t)} : t∈[M] }, where f^{(t)} is the polynomial obtained from f by setting* x{ij}↦0 for every (i,j)∉S_t. CONJECTURE: there exist absolute constants C₁,C₂>0 with C₂>4C₁·log₂ n / 1 such that for every n and every M with n ≤ M ≤ ⌊n^{1.5}⌋: (a) for every padded determinant g(x)=ℓ(x)^{n−m}·det_m(L(x)) with m≤⌊n^{1.5}⌋, ℓ a linear form and L an m×m matrix of linear forms in the x_{ij}, ρ_D(g) ≤ C₁·m·log₂ n; (b) ρ_D(perm_n) ≥ C₂·M. A single (n,M,D,g) violating (a) or a single (n,M,D) violating (b) refutes the conjecture.

Rationale (proposer's reasoning):

> NW designs probe disjoint-up-to-log-overlap windows of variables, so the M restrictions act like 'approximately independent samples' of the polynomial's syntactic content. A padded determinant compresses through only O(m²) hidden linear forms, so its NW-restrictions are forced to live in an O(m·polylog) dimensional subspace; the permanent has no such linear-form bottleneck, and its restrictions should be near-linearly-independent across NW slots. Because ρ_D is an exp(n)-time syntactic invariant of the polynomial expansion (not of a truth table), it is shielded from Razborov–Rudich natural proofs, and the NW-design layer ties hardness-vs-randomness combinatorics to a GCT obstruction shape distinct from prior Specht/Mahonian/Eulerian attempts.

Taxonomy category: `NISAN_WIGDERSON_DESIGNS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `202ac3d08e8f70f3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across n∈{4,5,6} and M∈{n,⌊n^1.25⌋,⌊n^1.5⌋} with 30 seeds each (270 trials), every seed must satisfy ρ_D(perm_n) ≥ 0.5·M AND ρ_D(padded det_m) ≤ 4·m·log₂ n AND gap=ρ_D(perm)−ρ_D(pad)>0. Support requires 100% of seeds meet all three; one violation falsifies.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Nisan-Wigderson design determinantal complexity permanent padded` - `combinatorial design restriction polynomial geometric complexity theory orbit closure` - `permanent determinant lower bound restriction dimension Nisan-Wigderson`

Top relevant hits considered: - [http://arxiv.org/abs/2202.13016v1] Bounds on Determinantal Complexity of Two Types of Generalized Permanents - [http://arxiv.org/abs/1007.3804v4] Symmetric Determinantal Representation of Formulas and Weakly Skew Circuits - [http://arxiv.org/abs/math/0106076v3] A determinantal formula for the Hilbert series of one-sided ladder determinantal rings

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import sys
import random
import math
import itertools
from collections import defaultdict

def matrix_mult(A, B):
    n = len(A)
    m = len(B[0])
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(len(B)):
```

```
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_rank(matrix):
    n = len(matrix)
    rank = 0
    for col in range(n):
        if rank >= n:
            break
        pivot = rank
        while pivot < n and matrix[pivot][col] == 0:
            pivot += 1
        if pivot == n:
            continue
        matrix[rank], matrix[pivot] = matrix[pivot], matrix[rank]
        for i in range(rank + 1, n):
            factor = matrix[i][col] / matrix[rank][col]
            for j in range(col, n):
                matrix[i][j] -= factor * matrix[rank][j]
        rank += 1
    return rank

def build_nw_design(n, M, seed):
    random.seed(seed)
    max_intersection = math.ceil(math.log2(n))
    design = []
    attempts = 0
    max_attempts = 1000
    while len(design) < M and attempts < max_attempts:
        subset = random.sample(range(n * n), n)
        valid = True
        for s in design:
            if len(set(subset) & set(s)) > max_intersection:
                valid = False
                break
        if valid:
            design.append(subset)
        attempts += 1
    if len(design) < M:
        return None
    return design

def perm_n(n):
    terms = list(itertools.permutations(range(n)))
    poly = defaultdict(int)
    for term in terms:
        poly[term] = 1 if sum(1 for i in range(n) if term[i] < i) % 2 == 0 else -1
    return poly

def padded_det(n, m, seed):
    random.seed(seed)
    L = [[random.randint(0, 1) for _ in range(m)] for _ in range(m)]
    ell = [random.randint(0, 1) for _ in range(n * n)]
    poly = defaultdict(int)
    for term in itertools.product(range(2), repeat=n * n):
        if sum(term) == n - m:
```

```
            sign = 1
            for i in range(n * n):
                if term[i] == 1:
                    sign *= ell[i]
            det_sign = 1
            for i in range(m):
                for j in range(m):
                    det_sign *= L[i][j]
            poly[term] = sign * det_sign
    return poly

def restrict_poly(poly, S):
    restricted = defaultdict(int)
    for term, coeff in poly.items():
        if all(i in S for i in range(len(term)) if term[i] != 0):
            restricted[term] = coeff
    return restricted

def run_trial(seed):
    n_values = [4, 5, 6]
    M_values = []
    for n in n_values:
        M_values.append([n, int(n**1.25), int(n**1.5)])
    results = []
    for n_idx, n in enumerate(n_values):
        for M in M_values[n_idx]:
            random.seed(seed)
            design = build_nw_design(n, M, seed)
            if design is None:
                continue
            perm = perm_n(n)
            m = int(n**1.5)
            det = padded_det(n, m, seed)
            perm_restrictions = []
            det_restrictions = []
            for S in design:
                perm_restrictions.append(restrict_poly(perm, S))
                det_restrictions.append(restrict_poly(det, S))
            all_terms = set()
            for r in perm_restrictions + det_restrictions:
                all_terms.update(r.keys())
            all_terms = sorted(all_terms)
            term_to_idx = {term: idx for idx, term in enumerate(all_terms)}
            perm_matrix = [[0 for _ in range(len(all_terms))] for _ in range(M)]
            det_matrix = [[0 for _ in range(len(all_terms))] for _ in range(M)]
            for i, r in enumerate(perm_restrictions):
                for term, coeff in r.items():
                    perm_matrix[i][term_to_idx[term]] = coeff
            for i, r in enumerate(det_restrictions):
                for term, coeff in r.items():
                    det_matrix[i][term_to_idx[term]] = coeff
            perm_rank = matrix_rank(perm_matrix)
            det_rank = matrix_rank(det_matrix)
            gap = perm_rank - det_rank
            conjecture_holds = (perm_rank >= 0.5 * M and det_rank <= 4 * m * math.log2(n) and gap
            counterexample = ""
```

```
            if not conjecture_holds:
                counterexample = f"n={n}, M={M}, perm_rank={perm_rank}, det_rank={det_rank}, gap={
            results.append({
                "n": n,
                "M": M,
                "metric_name": "gap",
                "metric_value": gap,
                "instances_tested": M,
                "conjecture_holds": conjecture_holds,
                "counterexample": counterexample
            })
    if not results:
        return {
            "metric_name": "gap",
            "metric_value": 0,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "No valid design found"
        }
    return results[0]

if __name__ == "__main__":
    seeds = sys.argv[1:] if len(sys.argv) > 1 else [random.randint(1, 1000) for _ in range(30)]
    seeds = [int(seed) for seed in seeds]
    metric_values = []
    conjecture_holds = []
    counterexamples = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        metric_values.append(result["metric_value"])
        conjecture_holds.append(result["conjecture_holds"])
        if result["counterexample"]:
            counterexamples.append((seed, result["counterexample"]))
    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values)
    support_fraction = sum(conjecture_holds) / len(conjecture_holds)
    if counterexamples:
        first_failing_seed, first_counterexample = counterexamples[0]
        print(f"RESULT: FALSIFIED counterexample=\"{first_counterexample}\" first_failing_seed={fi
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4a3a7c2.py", line 163, in <module>
    result = run_trial(seed)
```

```
            ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4a3a7c2.py", line 130, in run_trial
    perm_rank = matrix_rank(perm_matrix)
                ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4a3a7c2.py", line 33, in matrix_rank
    while pivot < n and matrix[pivot][col] == 0:
                        ~~~~~~~~~~~~~~^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with an IndexError in matrix_rank before any trials completed, so no data was produced to evaluate the pre-registered support or falsification conditions. | next: Fix the matrix_rank indexing bug (likely a row/column dimension mismatch when building perm_matrix) and rerun the full 270-trial sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	157205
2	preregistration	claude_max	opus	0	0	7817
3	novelty	claude_max	opus	0	0	4527
4	novelty	claude_max	opus	0	0	7577
5	test_gen	mistral	codestral-latest	0	0	313952
6	test_gen	mistral	codestral-latest	0	0	320300
7	test_gen	mistral	codestral-latest	0	0	313102
8	test_gen	mistral	codestral-latest	0	0	310664
9	judge	claude_max	opus	0	0	5712

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1440854 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/0cac01ce9ab0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0cac01ce9ab0.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0cac01ce9ab0.tar.gz` (if generated)